FINOOCEAN CRM

Codebase Audit Report

Generated: 2026-08-18

Scope: Full-stack audit (Angular frontend, Laravel backend, MySQL database)

Project Overview

Summary based on repository inspection (key files referenced).

Project architecture

- Monorepo with Angular frontend and Laravel backend.
  - Frontend: frontend/
  - Backend: backend/
  - Public assets: public/

Frontend stack

- Angular (TypeScript). Node.js-based toolchain and dependencies under frontend/package.json.
- Angular CLI cache present: frontend/.angular/

Backend stack

- Laravel (PHP) application in backend/.
- Authentication: Laravel Sanctum (see backend/config/sanctum.php).
- Policies and services implement business logic (examples: backend/app/Policies/LoanLeadPolicy.php, backend/app/Services/LeadService.php).

Database

- Migrations under backend/database/migrations/.
- Primary lead table: backend/database/migrations/2024_01_03_000000_create_loan_leads_table.php

Folder structure (high-level)

- backend/
  - app/ (Models, Http, Services, Policies)
  - routes/ (api.php)
  - database/ (migrations, seeders, factories)
  - config/
- frontend/
  - src/ (Angular app)
  - package.json

Dependencies

- Backend: composer-managed packages (see backend/composer.json, backend/composer.lock).
- Frontend: npm/yarn packages under frontend/package.json and large node_modules folder.

Notes and quick observations

- The backend implements a phased development plan (comments mention "Phase 9" for lead workflow).
- Business rules (assignment, visibility, matching) are enforced across repository, services and policies rather than purely in controllers.
- Several enums are used for typed domains: backend/app/Enums/ (UserRole, LeadStatus, DocumentType, LoanType).

References

- API routes: backend/routes/api.php
- Lead controller: backend/app/Http/Controllers/Api/V1/LeadController.php
- Lead service: backend/app/Services/LeadService.php
- Loan lead model: backend/app/Models/LoanLead.php

(End of overview)

User Management Audit

This document inspects authentication, authorization, roles, permissions, hierarchy and department assignment.

Authentication

- Laravel Sanctum is used for API token/session authentication. See backend/routes/api.php endpoints for auth: login, logout, me.
- Auth controller: backend/app/Http/Controllers/Api/V1/Auth/AuthController.php

Authorization

- Role/permission middleware exists on routes (e.g., middleware('role:manager') and permission middleware like 'permission:users.view'). See backend/routes/api.php.
- Fine-grained object-level authorization uses policies. Example: backend/app/Policies/LoanLeadPolicy.php.

Roles

- Implemented roles (enum): backend/app/Enums/UserRole.php
  - super_admin
  - manager
  - relationship_manager
  - employee
  - connector
  - banker

Permissions

- Permission middleware used for user CRUD routes (see users group in backend/routes/api.php).
- Application appears to use a simple permission check via middleware strings (likely implemented in a package or custom middleware).

Hierarchy

- Hierarchy fields on User model: manager_id, relationship_manager_id, employee_id.
  - User model: backend/app/Models/User.php
- UserRole enum documents the intended parent roles (see allowedParents()).

Department assignment

- Departments (loan product domains) are implemented as loan_type on leads rather than a separate department entity.

Feature matrix

Feature	Status
Super Admin	✓
Manager	✓
RM	✓
Employee	✓
Telecaller	✓ (see roles/guards - telecaller exists in frontend role lists)
Banker	✓

Notes and issues

- Roles implemented match project requirement list.
- Permissions are used but require runtime check of middleware mapping (not all permission strings are enumerated in code search). Investigate middleware implementation for `permission:` and `role:` enforcement.

References

- backend/app/Models/User.php
- backend/app/Enums/UserRole.php
- backend/routes/api.php
- backend/app/Policies/LoanLeadPolicy.php

Database Audit

This file enumerates tables, relationships, indexes, foreign keys and normalization observations relevant to lead workflow.

Implemented tables (selected)

- loan_leads: migration
  - columns: reference, applicant_name, applicant_email, applicant_phone, loan_type, loan_amount, tenure_months, monthly_income, credit_score, employment_type, city, state, pincode, status, created_by, connector_id, notes, timestamps, soft deletes
  - later migrations add assigned_employee_id, assigned_relationship_manager_id, assigned_manager_id, updated_by, assigned_banker_id
- documents: stores uploaded documents (document_type, path, uploaded_by) migration
- lead_assignments: assignment history migration
- lead_status_history: status transitions migration
- lead_matches: matching results migration
- users: standard user table (see backend/database/migrations — typical Laravel users migration present)

Relationships

- loan_leads.created_by -> users.id (nullable, nullOnDelete)
- loan_leads.connector_id -> users.id
- loan_leads.assigned_* -> users.id (cascade null on delete)
- documents.loan_lead_id -> loan_leads.id (cascade on delete)
- lead_assignments.loan_lead_id -> loan_leads.id (cascade)
- lead_status_history.loan_lead_id -> loan_leads.id (cascade)
- lead_matches.loan_lead_id -> loan_leads.id (cascade)

Indexes & constraints

- Indexes exist on loan_type and status (migrations show ->index()).
- Foreign keys created by constrained() calls in migrations.

Normalization

- Data model is normalized for leads: separate history tables (assignments, status_history, matches, documents).
- Loan types and user roles are represented as enums (application-level). No separate domain tables for loan products; bank criteria are in bank_criteria table with eligibility_rules JSON.

Missing / Incomplete items

- No dedicated departments table; "department" concept is implemented implicitly via loan_type values. If strict department-level visibility required, a department table or stronger loan_type taxonomy may be needed.
- Several universal lead form fields requested in the audit (e.g., alternative_phone, gender, date_of_birth, company_name, existing_emi, annual_turnover, property_value, property_type) are not present in `loan_leads` migration and model.

Implemented tables / Missing tables / Unused tables (summary)

- Implemented tables: loan_leads, documents, lead_assignments, lead_status_history, lead_matches, bank_criteria, users, notifications.
- Missing tables: explicit departments table (if required by spec), telecaller_call_logs (no dedicated call history table found named as such), document_versioning table (no version history for documents beyond soft deletes).
- Unused tables: none obvious from quick scan; vendor and node_modules directories contain many files but are not application tables.

Incorrect relationships

- None structurally incorrect in migrations. Relationship enforcement and visibility rely heavily on repository query scopes, not purely on DB-level constraints; ensure repository scopes match intended visibility.

References

- backend/database/migrations/
- backend/app/Models/LoanLead.php

(End of DB audit)

API Audit

This document enumerates major API endpoints, controllers, authentication and access rules found during code inspection.

Routing

- All API v1 routes are defined in: backend/routes/api.php.
- Routes are grouped by role prefixes and guarded with `auth:sanctum` and `auth.active` middleware.

Key endpoints (non-exhaustive)

API endpoint	Method	Controller	Authentication	Role access	Status
/api/v1/auth/login	POST	AuthController@login	unauthenticated (throttled)	public	Implemented
/api/v1/auth/me	GET	AuthController@me	sanctum	authenticated	Implemented
/api/v1/leads	GET	LeadController@index	sanctum	policy: viewAny	Implemented
/api/v1/leads	POST	LeadController@store	sanctum	policy: create	Implemented
/api/v1/leads/{lead}/assign	POST	LeadController@assign	sanctum	policy: assign	Implemented
/api/v1/leads/{lead}/status	POST	LeadController@updateStatus	sanctum	policy: changeStatus	Implemented
/api/v1/leads/{lead}/match	POST	LeadController@match	sanctum	policy: match	Implemented
/api/v1/documents	POST	DocumentController@store	sanctum	permission/middleware + policy	Implemented
/api/v1/users	GET/POST/PUT/DELETE	UserController	sanctum	permission:*	Implemented with permission middleware

Controllers

- LeadController: backend/app/Http/Controllers/Api/V1/LeadController.php
- DocumentController: backend/app/Http/Controllers/Api/V1/DocumentController.php
- UserController: backend/app/Http/Controllers/Api/V1/UserController.php

Services & Repositories

- Lead workflow is implemented in service layer: backend/app/Services/LeadService.php
- A repository interface is used for loan lead persistence: backend/app/Repositories/Contracts/LoanLeadRepositoryInterface.php and concrete implementations under app/Repositories.

Policies & Middleware

- LoanLeadPolicy enforces who may create, update, assign, change status etc.: backend/app/Policies/LoanLeadPolicy.php
- Routes use role middleware (e.g., `->middleware('role:manager')`) and permission middleware (e.g., `permission:users.view`). Implementation of these middleware should be reviewed (search for RoleMiddleware/PermissionMiddleware in app/Http/Middleware or registered packages).

Status / Issues

- Most endpoints for lead lifecycle are implemented and wired to service layer.
- Policy `changeStatus` allows employees and bankers to change the status of leads they are assigned to (LoanLeadPolicy::changeStatus). This contradicts the business-rule requirement that only Super Admin, Manager, Relationship Manager may change status. See `backend/app/Policies/LoanLeadPolicy.php` lines 59-66.

Recommendations

- Review LoanLeadPolicy::changeStatus and align with the business requirement (disallow Employee/Banker unless explicitly permitted).

- Verify middleware and permission mapping for role and permission strings to ensure runtime enforcement.

References

- backend/routes/api.php
- backend/app/Http/Controllers/Api/V1/LeadController.php
- backend/app/Policies/LoanLeadPolicy.php
- backend/app/Services/LeadService.php

Frontend Audit

This section summarizes the Angular frontend structure, routes, components, services and state management observed in the repository.

Location

- Frontend app: frontend/

Pages & Routes

- Routing is defined in the Angular app (inspect src/app/app-routing.module.ts in frontend). Key pages include login, dashboard(s) for roles, leads list/detail, lead form, documents upload.

Components & Services

- Services call backend APIs (look in src/app/services or src/app/api) and consume the metadata endpoint for form catalogue values: `/api/v1/leads/metadata` (LeadController::metadata).
- Forms rely on the backend to supply loan types, statuses, and employment types.

Guards & State

- Guards use stored authentication state (probably via a token stored by an Auth service). The backend uses Sanctum; confirm frontend auth flow (sanctum cookie or token-based) in [frontend/src/app/auth] or similar.

Implemented pages

- Login page
- Role-specific dashboards (super-admin, manager, relationship-manager, banker, connector, employee)
- Lead listing and detail pages
- Lead create / edit form (consumes metadata endpoint)

Missing/Incomplete items / Broken pages

- No explicit Telecaller module was discovered in backend routes; frontend may include a telecaller-only UI page but backend role/middleware and policies must support telecaller workflows. Search frontend for "telecaller" occurrences to confirm.
- Verify file upload flows: DocumentController endpoints exist; frontend should map the document types to the backend DocumentType enum.

Unused Components

- Not exhaustively listed here — run a targeted static analysis of src/app to find unused components.

References

- frontend/
- Lead metadata endpoint: backend/app/Http/Controllers/Api/V1/LeadController.php#metadata

(End of frontend audit)

Lead Workflow Audit

This document describes lead creation, visibility, assignment, reassignment, ownership and history mechanisms.

Lead creation

- Endpoint: POST /api/v1/leads -> LeadController@store
- Anyone active may create a lead (LoanLeadPolicy::create returns is_active). The service stamps created_by and initial status `new`.

Lead visibility

- Visibility is enforced at repository scope: LeadService->findVisible uses LoanLeadRepositoryInterface->findVisible and paginateVisible implementations. See repository implementations under backend/app/Repositories/.
- Controller authorizes view via LoanLeadPolicy::view/viewAny but visibility scope prevents access to leads outside actor's subtree.

Lead assignment & reassignment

- Assignment endpoint: POST /api/v1/leads/{lead}/assign -> LeadController@assign
- Assignment authorization: LoanLeadPolicy::assign allows SuperAdmin, Manager, RelationshipManager.
- LeadService->assign enforces further rules (guardAssignment) and cascades assigned_employee/relationship_manager/manager fields.
- Reassignment: assignment creates LeadAssignment history entries and deactivates prior active assignment records.

Lead status changes

- Endpoint: POST /api/v1/leads/{lead}/status -> LeadController@updateStatus
- Policy: LoanLeadPolicy::changeStatus currently allows SuperAdmin, Manager, RelationshipManager, and ALSO allows Employee and Banker when they are the assigned party.
- Business requirement (audit prompt) states only Super Admin, Manager, Relationship Manager can change status — codebase deviates (allows assigned Employee/Banker). Marked as INCORRECT (see recommendations).

Lead ownership & transfer

- Ownership is recorded on loan_leads via assigned_employee_id, assigned_relationship_manager_id, assigned_manager_id, assigned_banker_id and created_by.
- LeadAssignment table stores assignment history; LeadStatusHistory stores status transition history; Document table stores uploads.

Lead history

- LeadController@show loads statusHistory, assignments and matches to produce a complete audit trail.
- Models: LeadAssignment, LeadStatusHistory and Document are used for history/trace.

Workflow diagram (textual)

- Create (any active role) -> created_by, status=new -> notifications to assigned RM/Manager
- Assign (SuperAdmin/Manager/RM) -> cascades assigned_* fields, assignment history entry, status becomes `assigned` if was `new` -> notify assignee
- Change status (per policy) -> leads->changeStatus records status history and notifies stakeholders
- Match (non-banker/connector roles) -> matching service invoked to produce bank matches

References

- backend/app/Services/LeadService.php
- backend/app/Policies/LoanLeadPolicy.php
- backend/app/Http/Controllers/Api/V1/LeadController.php

(End of lead workflow audit)

Telecaller Audit

Scope: PD workflow (pre-sales / telecaller), customer verification, status updates and call history.

Findings

- Backend routes and policies do not explicitly mention a Telecaller role in lead-specific endpoints. The UserRole enum includes roles: `connector`, `banker`, `employee`, `relationship_manager`, `manager`, `super_admin` — no explicit `telecaller` enum value.
- The API route groups mention phases for Employee, RM, Manager, Banker and Connector; there is no dedicated `telecallers` prefix in `routes/api.php`.

Customer verification

- No dedicated telecaller verification controller or call-log table was found named `telecaller_calls` or similar. Document uploads and notes exist and can be used to record verification steps.

Status updates

- Telecallers are not explicitly permitted to change status by policy (LoanLeadPolicy::changeStatus). The policy allows status changes by SuperAdmin, Manager, RM and — as currently implemented — by Employees/Bankers for assigned leads.

Call history

- No dedicated call-history table or model found in the migrations (no `call_logs` or `telecaller_calls`). Suggest adding a call logs table if telecaller detailed workflows are required.

Conclusion

- Telecaller functionality appears to be planned at a UI level, but backend support (role, routes, call-history persistence) is missing or incomplete.

Recommendations

- If telecaller workflows are required: add a Telecaller role to UserRole enum, create API endpoints and persistence for call logs, and clarify what telecallers may view/modify (create leads, add notes, verify identity).

References

- backend/app/Enums/UserRole.php
- backend/routes/api.php

Banker Audit

Banker-related features: bank criteria, lead requests, assignment and document requests.

Bank criteria

- Bank criteria table and controller exist: endpoints under `/api/v1/banks` are implemented in BankCriteriaController and migration in `database/migrations`.
- Bank criteria include `eligibility_rules` JSON (see 2024_01_08_000000_extend_loan_workflow_tables.php).

Lead requests & assignment

- Bankers are a standalone role (UserRole::Banker). They can have leads assigned via assigned_banker_id (added 2026 migration).
- LeadService::assign allows SuperAdmin/Manager/RM to assign leads to bankers as a special case (guardAssignment), so bank assignment flow is supported.

Document requests & loan processing

- Bankers may upload documents if assigned (DocumentController upload is guarded by uploadDocument policy which allows banker when update() returns true). Confirm DocumentController and DocumentService behavior.

Recommendations

- Ensure banker-specific actions (loan processing steps) are represented in API and UI (e.g., banker dashboards, status transitions unique to banker workflows).

References

- backend/app/Enums/UserRole.php
- backend/app/Services/LeadService.php
- backend/database/migrations/2026_08_14_100300_add_assigned_banker_id_to_loan_leads_table.php

Matching Engine Audit

This section inspects the matching algorithm and how bank criteria are used to produce matches.

Implementation

- Matching service: backend/app/Services/LeadMatchingService.php (used by LeadService->runMatching).
- Lead matches are stored in `lead_matches` table (migration: 2024_01_03_000003_create_lead_matches_table.php).
- Bank criteria stored in `bank_criteria` table with `eligibility_rules` JSON; matching engine evaluates eligibility_rules against lead attributes.

Criteria comparison

- Eligibility rules reside as JSON and are interpreted by the matching service (read the service to confirm supported operators and scoring rules).

Score calculation & ranking

- The matching service computes a match_score and reasons; stored fields include match_score, reasons and status on each match record.
- Rank is returned by LeadController@show in the `matches` payload (match_score and reasons included).

Assignment

- Matching is independent from assignment. A separate assignment flow assigns leads to users/bankers after matching.

Recommendations

- Validate matching logic against all intended bank eligibility criteria and unit-test edge cases.
- Externalize complex eligibility rules to a formal DSL or rule engine if they grow beyond simple JSON comparison.

References

- backend/app/Services/LeadService.php
- backend/app/Services/LeadMatchingService.php
- backend/database/migrations/2024_01_03_000003_create_lead_matches_table.php

Document Management Audit

This document inspects file uploads, storage, types and versioning.

File uploads

- Documents persisted in `documents` table with `document_type`, `path`, `uploaded_by` and `status`.
- Document types are enumerated in backend/app/Enums/DocumentType.php and include: aadhaar, pan, bank_statement, income_proof, other.

Storage

- Files are stored under Laravel storage (examples under `backend/storage/app/private/lead-documents/`). Look for storage disk configuration in `backend/config/filesystems.php`.

Additional documents and version history

- There is no separate document versioning table; documents are soft-deleted when removed. No explicit version history feature is present beyond storing multiple rows per lead.

Missing items versus spec

- Required doc types listed in the audit (Front Aadhaar, Back Aadhaar, PAN Card, 6-Month Bank Statement, Electricity Bill):
  - Aadhaar and PAN: supported (DocumentType::Aadhaar, DocumentType::Pan)
  - Bank statement: supported (bank_statement) but no explicit enforcement of 6-month requirement at DB level
  - Electricity bill: not a distinct DocumentType (could be uploaded as "other").

Recommendations

- Add electricity_bill to DocumentType enum if a distinct type is required.
- Consider document versioning if legal/audit capabilities are required (store parent_id or version column).

References

- backend/app/Enums/DocumentType.php
- backend/database/migrations/2024_01_03_000005_create_documents_table.php
- Example stored file: backend/storage/app/private/lead-documents/

Notifications Audit

Overview of notifications, emails, alerts and activity logs.

Notifications

- NotificationService is used in LeadService to notify stakeholders on lead creation, assignment and status changes (see backend/app/Services/LeadService.php).
- NotificationController exists and routes notifications endpoints (see backend/routes/api.php).

Emails

- Email notification mechanism likely implemented via NotificationService; search for mail channel usage in Notifications classes under app/Notifications.

Alerts & Activity logs

- Application uses Log::info and Log::error in LeadController during creation to record success/failure traces. Activity audit for leads is captured via status history and assignment records stored in dedicated tables.

Recommendations

- Verify NotificationService implementation to confirm channels (email, in-app, SMS) and that users' preferences (email_notifications & sms_notifications on User) are honored.

References

- backend/app/Services/NotificationService.php (inspect implementation)
- backend/app/Services/LeadService.php (notification usage)
- backend/app/Models/Notification.php (if present)

Security Audit

This section inspects authentication, API security, file security, JWT (or token) usage and input validation.

Authentication & tokens

- Laravel Sanctum is used for API authentication (routes use `auth:sanctum`). Inspect backend/config/sanctum.php and backend/app/Http/Controllers/Api/V1/Auth/AuthController.php.

Role permissions & API security

- Route-level role middleware (e.g., `->middleware('role:manager')`) and permission middleware (e.g., `permission:users.view`) guard routes.
- Policies provide object-level authorization (LoanLeadPolicy, etc.).

File security

- Documents are stored in `storage/app/private/lead-documents/` and downloaded via DocumentController@download which should enforce authorization. Verify the controller enforces `uploadDocument` and/or `download` policies.

Input validation

- Controller actions use Form Requests (e.g., StoreLeadRequest, UpdateLeadStatusRequest, AssignLeadRequest) for validation.
- Ensure the validation rules cover all required lead fields and document type whitelists (Document upload request uses Rule::in(DocumentType::values())).

JWT

- The project uses Sanctum, not JWT. No JWT implementation found.

Vulnerabilities / Issues found

- Policy divergence: LoanLeadPolicy::changeStatus allows Employee and Banker to change status of assigned leads which conflicts with the stated business rule. This is an authorization policy issue, not a token problem.
- Ensure file download endpoints verify lead visibility before streaming files to the requester (DocumentController should use repository scope & policy checks).

Recommendations

- Harden DocumentController@download to ensure strict visibility and policy checks (no direct path leakage). Use signed temporary URLs for downloads if required.
- Review authentication cookie configuration (sameSite, secure) if using Sanctum with SPA cookies.

References

- backend/routes/api.php
- backend/app/Policies/LoanLeadPolicy.php
- backend/app/Http/Requests/ (validation requests)
- backend/app/Http/Controllers/Api/V1/DocumentController.php

Missing Features

This table marks completion status for major modules based on repository inspection.

Module	Status
Authentication	COMPLETE
User Management	COMPLETE
Lead Management	PARTIAL
Telecaller	MISSING/PARTIAL
Matching Engine	PARTIAL
Banker	PARTIAL
Notifications	PARTIAL
Analytics	MISSING

Notes

- Lead Management: core CRUD, assignment, matching, history are implemented, but Universal Lead Form fields required by the spec are missing (see database audit). Policy misalignment on status changes exists.
- Telecaller: backend persistence for call logs and explicit telecaller role endpoints absent; likely missing or UI-only.
- Matching Engine: implemented but needs verification and unit tests for complex eligibility rules.
- Analytics: no dedicated analytics module or dashboards discovered; reports endpoint has a dashboard route but scope is unclear (backend/routes/api.php).

References

- See other docs for specifics and required actions.

Implementation Roadmap

This roadmap lists critical, high, medium and low priority items to align the implementation with the business requirements provided.

Critical issues

1. Policy mismatch: Restricting status changes
    - What exists: LoanLeadPolicy::changeStatus currently allows Employees and Bankers to change status for leads assigned to them.
    - What's missing: Business rule requires only Super Admin, Manager, Relationship Manager may change lead status.
    - Required files: backend/app/Policies/LoanLeadPolicy.php
    - Required APIs: none (policy change only)
    - Required DB changes: none
    - Implementation steps:
        1. Modify LoanLeadPolicy::changeStatus to only allow SuperAdmin, Manager, RelationshipManager (remove Employee/Banker branches), or add explicit config toggle.
        2. Add regresion tests for status-change authorization.

High-priority issues 2. Universal Lead Form: missing fields

- What exists: loan_leads table and model include applicant_name, applicant_email, applicant_phone, loan_type, loan_amount, monthly_income, credit_score, employment_type, city, state, pincode, notes.
- What's missing: alternative_phone, gender, date_of_birth, company_name, existing_emi, annual_turnover, property_value, property_type, and existing-loan checklist flags (repayment_track, noc, foreclosure, soa, lod).
- Required files: migrations to add columns (database/migrations/...), updates to model [backend/app/Models/LoanLead.php], update request validation (StoreLeadRequest, UpdateLeadRequest), update frontend form (frontend/src/app/...)
- Required APIs: none new, extend existing POST /api/v1/leads and PUT /api/v1/leads/{id} to accept/validate fields
- Required DB changes: add new nullable columns to loan_leads via migration; consider JSON column for checklist if many boolean flags.
- Implementation steps:
    1. Create migration to add missing columns.
    2. Update LoanLead::$fillable and $casts.
    3. Update StoreLeadRequest/UpdateLeadRequest rules.
    4. Update frontend form to include fields and map to backend metadata if needed.

Medium-priority issues 3. Telecaller workflows and call history

- What exists: UI may reference telecaller flows; backend lacks dedicated persistence and role-specific routes.
- What's missing: telecaller role in backend (enum), call logs table, APIs to log calls, telecaller dashboard endpoints.
- Required files: migration for call_logs, model, controller, routes and policy; frontend pages/components.
- Implementation steps:
    1. Add Telecaller role to UserRole enum if required.
    2. Add call_logs migration and model.
    3. Add APIs under a telecallers prefix and policies.

4. Document type: Electricity Bill
    - Add `electricity_bill` to DocumentType enum and reflect it in frontend UI.

Low-priority issues 5. Document versioning

- Add versioning table/columns or implement copy+reference to track document uploads over time.

6. Analytics & reports
    - Implement or expand reports/dashboard endpoints and frontend visualizations.

For every change, ensure:

- Update Form Requests for validation
- Add tests (unit and integration) to cover authorization, repository visibility scopes and matching logic
- Update API docs / OpenAPI spec if present

(End of roadmap)